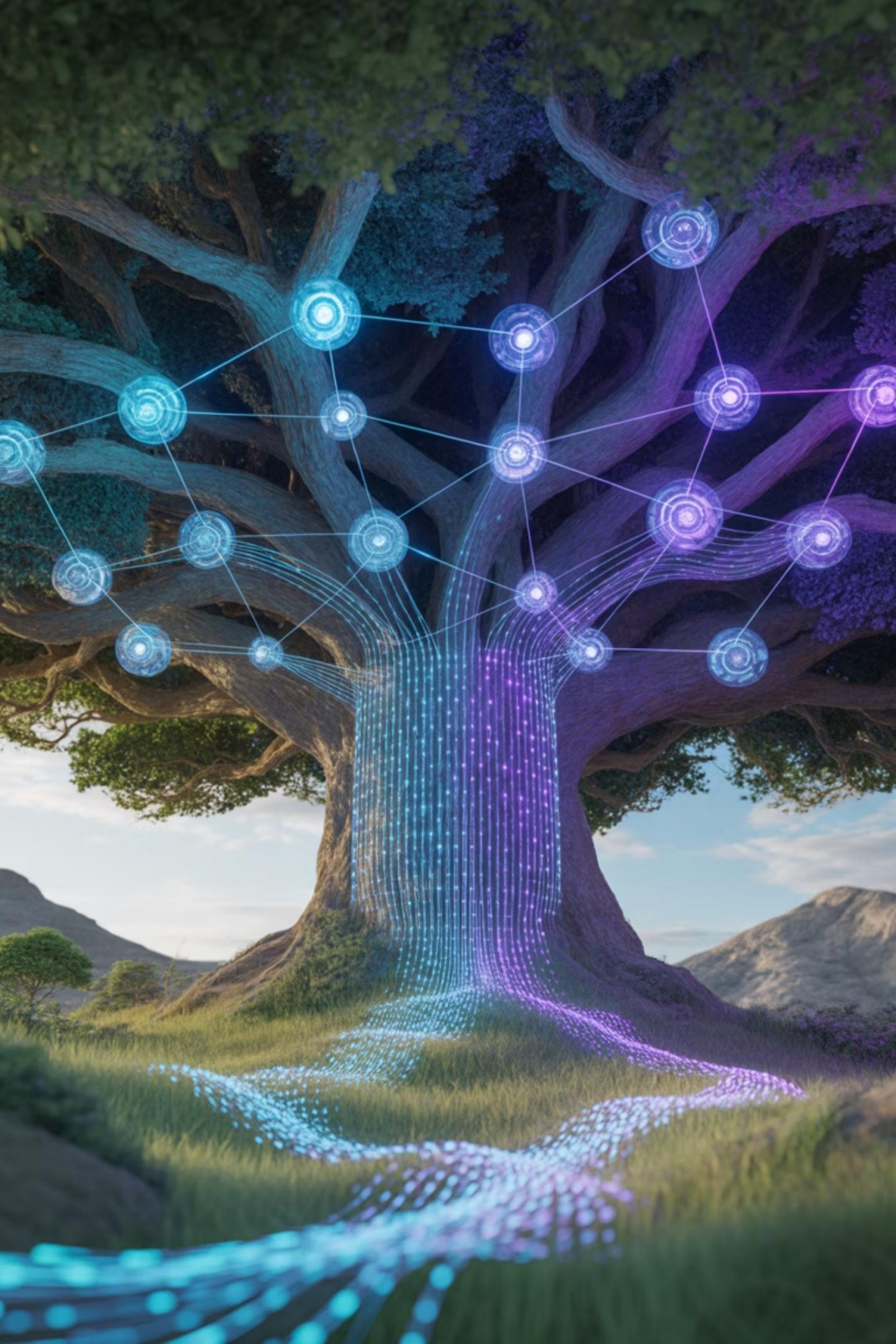


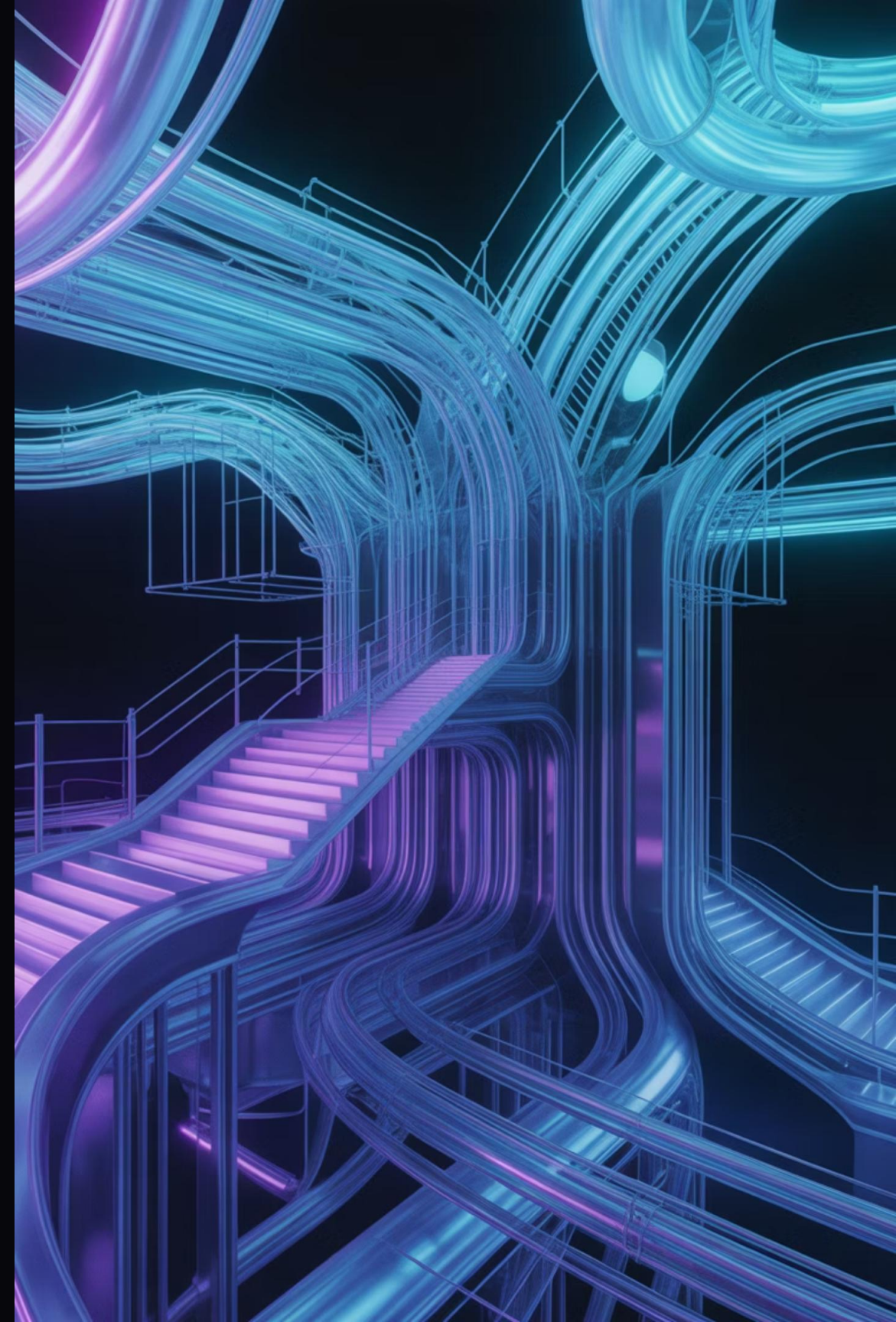


Conceitos, Implementações e Aplicações de Árvores

Uma jornada completa pelo universo das estruturas de dados hierárquicas, explorando desde fundamentos teóricos até aplicações práticas em sistemas reais.

Capítulo 1: Fundamentos e Conceitos Básicos de Árvores

Iniciamos nossa exploração compreendendo os conceitos fundamentais que sustentam uma das estruturas de dados mais versáteis e poderosas da ciência da computação.



O que é uma Árvore em Ciência da Computação?

Uma árvore é uma estrutura de dados hierárquica composta por nós conectados por arestas, onde cada conexão representa uma relação pai-filho. Diferentemente de estruturas lineares como arrays e listas encadeadas, as árvores organizam dados em múltiplos níveis, permitindo representações naturais de hierarquias.

A estrutura começa com um **nó raiz**, que não possui pai. A partir dele, ramificam-se **nós internos** que possuem filhos, e **folhas** que são nós terminais sem descendentes. Cada nó pode ter múltiplos filhos, e todos compartilham ancestrais comuns em níveis superiores.

Estruturas Lineares

Dados organizados sequencialmente em uma única dimensão

Estruturas Hierárquicas

Dados organizados em múltiplos níveis com relações pai-filho

Terminologia Essencial

1

Raiz

O nó superior da árvore, sem pai, ponto de partida para todas as operações

2

Folha

Nó terminal sem filhos, representa o fim de um caminho na estrutura

3

Grau do Nó

Número de filhos diretos que um nó possui na estrutura

4

Nível

Distância de um nó até a raiz, começando em 0 para a raiz

5

Altura

Comprimento do caminho mais longo da raiz até uma folha

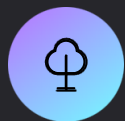
6

Profundidade

Distância de um nó específico até a raiz da árvore

Uma **floresta** é definida como um conjunto de árvores disjuntas, ou seja, múltiplas árvores que não compartilham nós entre si. Este conceito é útil em algoritmos que dividem estruturas complexas em componentes menores.

Tipos de Árvores



Árvore Ordinária

Cada nó pode ter qualquer número de filhos, sem restrições de quantidade



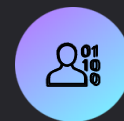
Árvore Completa

Todos os níveis estão completamente preenchidos, exceto possivelmente o último



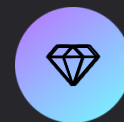
Árvore Inclínada

Cada nó possui apenas um filho, formando uma estrutura linear



Árvore Binária

Cada nó possui no máximo dois filhos: esquerdo e direito



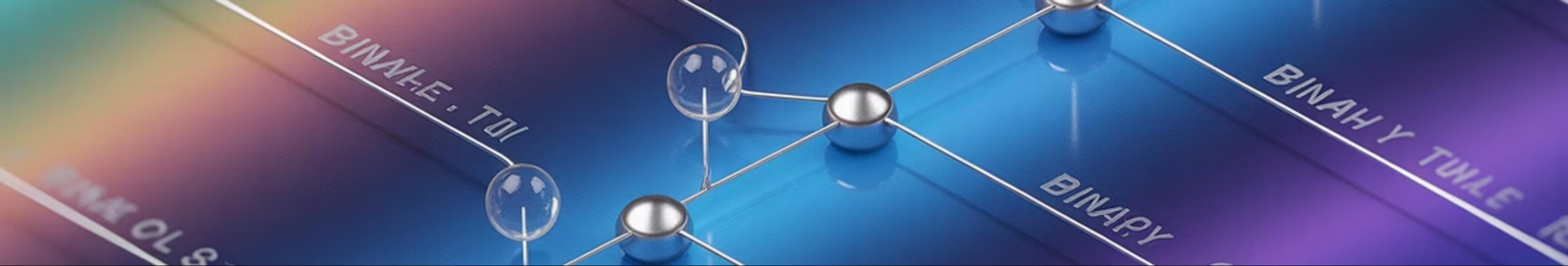
Árvore Perfeita

Todos os níveis estão completamente preenchidos, incluindo o último nível



Árvore Patológica

Degenerada em lista encadeada, perdendo eficiência de busca



Estrutura Hierárquica Visual

A visualização de uma árvore binária revela claramente a organização hierárquica dos dados, com cada nó conectado aos seus filhos por arestas direcionadas, formando uma estrutura que facilita operações de busca, inserção e remoção eficientes.

Recursão em Árvores

A recursão é o paradigma fundamental para trabalhar com árvores, pois a própria estrutura é naturalmente recursiva: uma árvore é composta por uma raiz e subárvores.

As três principais formas de percorrer uma árvore recursivamente são:

- **Pré-ordem:** visita raiz, depois subárvore esquerda, depois direita
- **Em-ordem:** visita subárvore esquerda, raiz, depois direita
- **Pós-ordem:** visita subárvore esquerda, direita, depois raiz

Pseudocódigo: Travessia Em-Ordem

```
função emOrdem(nó):    se nó é nulo:        retornar  
emOrdem(nó.esquerda)    imprimir(nó.valor)  
emOrdem(nó.direita)
```

Este código demonstra a elegância da recursão: a solução para a árvore inteira é definida em termos de soluções para suas subárvores.



Capítulo 2: Implementações Práticas de Árvores

Agora mergulhamos no mundo prático da implementação, transformando conceitos teóricos em código funcional que pode ser executado e testado.

Representação de Árvores em Código

Ponteiros e Referências

A forma mais natural de implementar árvores, onde cada nó contém referências diretas para seus filhos, permitindo navegação eficiente pela estrutura.

Representação com Arrays

Utiliza um array onde o filho esquerdo do nó em índice i está em $2i+1$ e o direito em $2i+2$.
Eficiente para árvores completas.

Listas de Adjacência

Cada nó mantém uma lista de seus filhos, flexível para árvores com grau variável e economiza espaço em árvores esparsas.

Exemplo em Python: Definição Básica

```
class No: def __init__(self, valor): self.valor = valor self.esquerda = None self.direita = None
```

Construindo uma Árvore Binária Simples em Python

Vamos construir uma árvore binária passo a passo, criando nós e estabelecendo as conexões entre eles. Este exemplo demonstra a implementação mais direta e intuitiva.

```
# Criando os nós da árvore
raiz = No(50)
raiz.esquerda = No(30)
raiz.direita = No(70)
raiz.esquerda.esquerda = No(20)
raiz.esquerda.direita = No(40)
raiz.direita.esquerda = No(60)
raiz.direita.direita = No(80)

# Função para visualizar a árvore
def imprimir_arvore(no, nivel=0, prefixo="Raiz: "):
    if no is not None:
        print(" " * (nivel * 4) + prefixo + str(no.valor))
        if no.esquerda or no.direita:
            imprimir_arvore(no.esquerda, nivel + 1, "Esq: ")
            imprimir_arvore(no.direita, nivel + 1, "Dir: ")

# Visualização
imprimir_arvore(raiz)
```

Este código cria uma árvore com 7 nós e uma função de visualização que exibe a estrutura hierárquica de forma indentada, facilitando a compreensão da organização dos dados.

Implementação de Travessias Recursivas

Código das Três Travessias

```
def pre_ordem(no):    if no:        print(no.valor, end=" ")
pre_ordem(no.esquerda)    pre_ordem(no.direita)def
em_ordem(no):    if no:        em_ordem(no.esquerda)
print(no.valor, end=" ")        em_ordem(no.direita)def
pos_ordem(no):    if no:        pos_ordem(no.esquerda)
pos_ordem(no.direita)    print(no.valor, end=" ")
```

Resultados com Nossa Árvore

Pré-ordem: 50 30 20 40 70 60 80

Em-ordem: 20 30 40 50 60 70 80

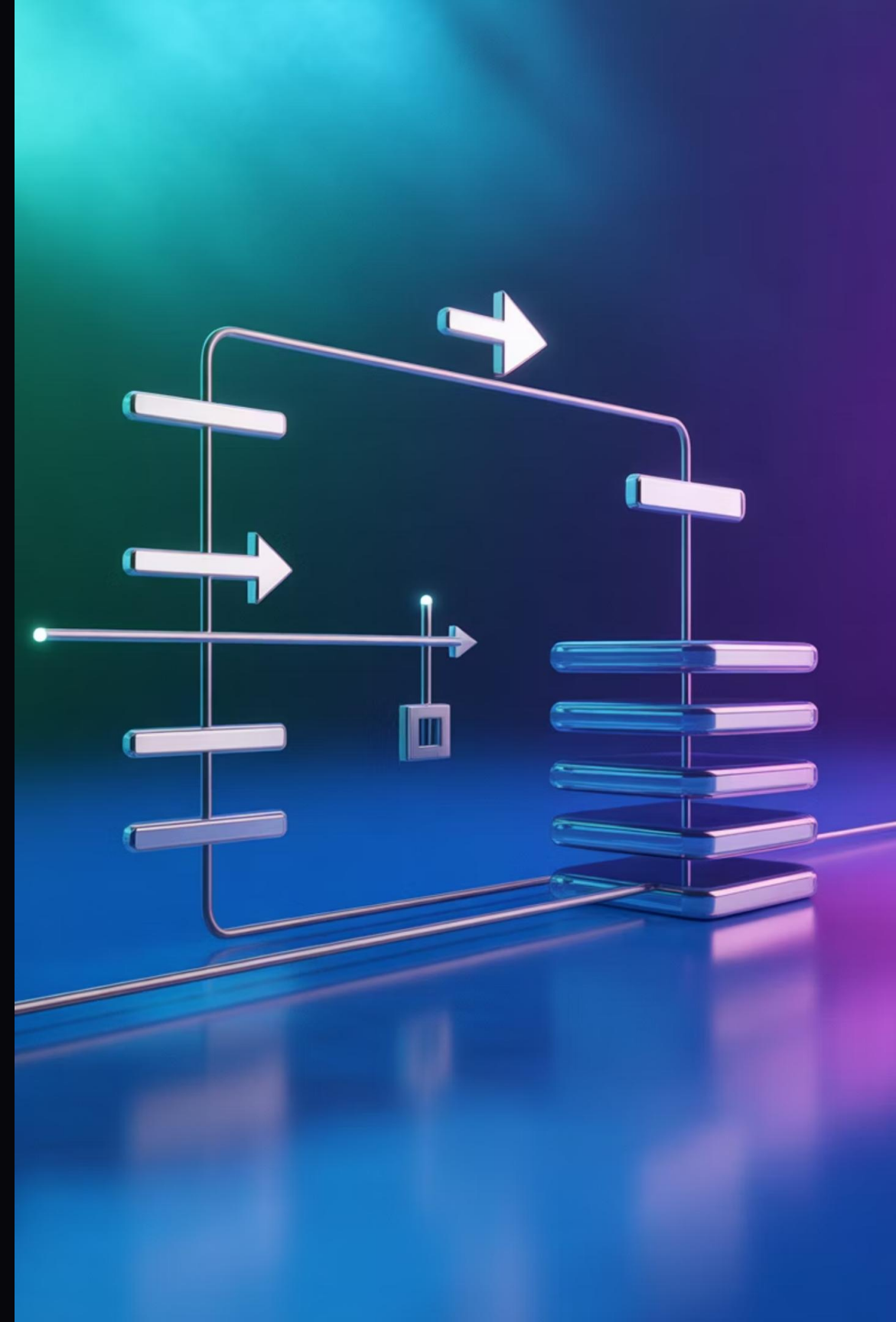
Pós-ordem: 20 40 30 60 80 70 50

Observe como a travessia em-ordem produz os valores em ordem crescente - esta propriedade é fundamental nas Árvores Binárias de Busca.

O fluxo das chamadas recursivas segue um padrão de empilhamento: cada chamada aguarda o retorno das chamadas internas antes de prosseguir, criando uma navegação em profundidade pela estrutura.

Recursão: O Coração das Árvores

A elegância da recursão reside em sua capacidade de decompor problemas complexos em casos mais simples, refletindo perfeitamente a natureza hierárquica das árvores.



Árvores Binárias de Busca (ABB)

Uma Árvore Binária de Busca é uma estrutura que mantém uma propriedade fundamental: para cada nó, todos os valores na subárvore esquerda são menores que o valor do nó, e todos na subárvore direita são maiores.

Inserção em ABB

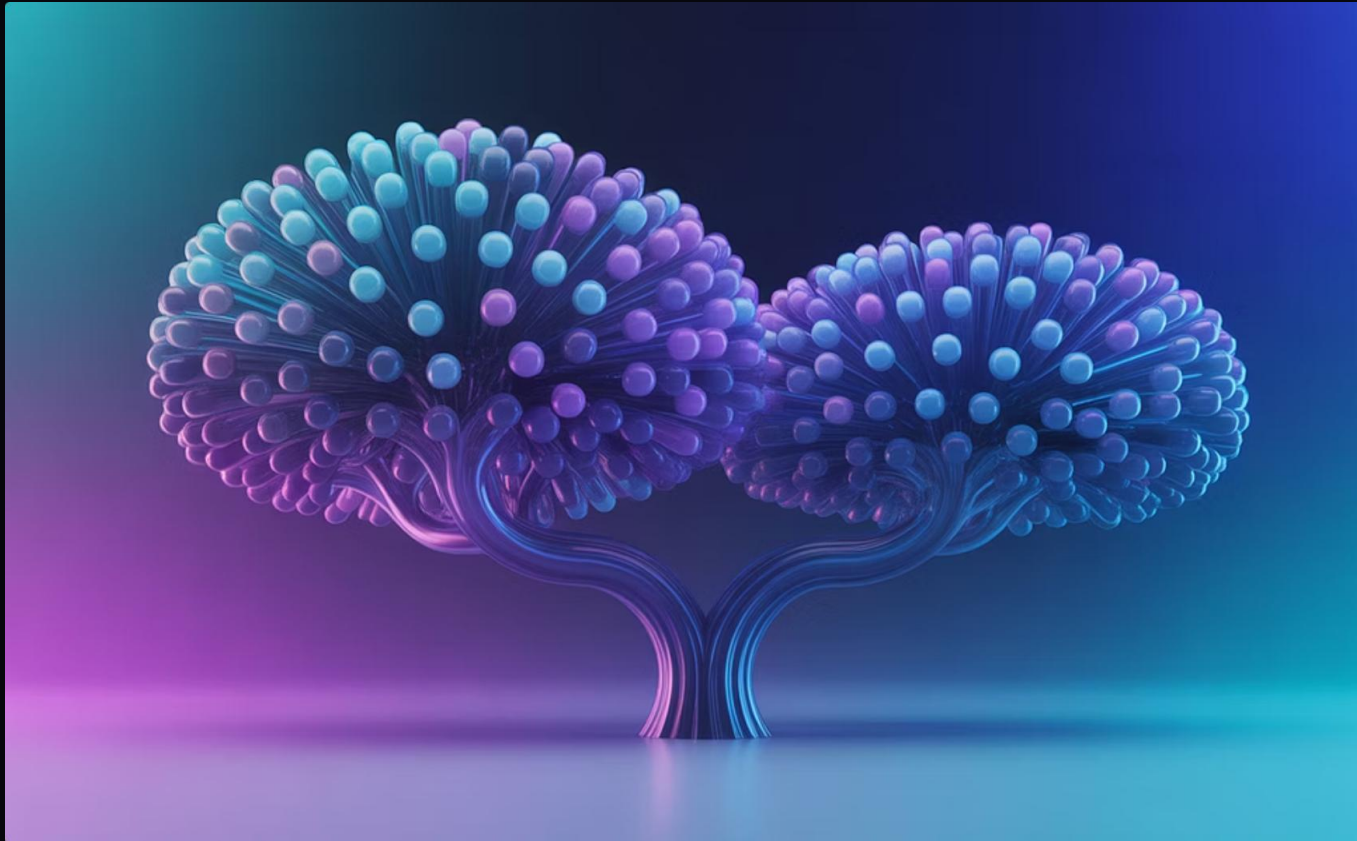
```
def inserir(raiz, valor):    if raiz is None:        return  
                             No(valor)        if valor < raiz.valor:            raiz.esquerda =  
inserir(                raiz.esquerda, valor            )        else:  
raiz.direita = inserir(                raiz.direita, valor            )  
return raiz
```

Busca em ABB

```
def buscar(raiz, valor):    if raiz is None:        return False  
if valor == raiz.valor:        return True    elif valor <  
raiz.valor:        return buscar(                raiz.esquerda, valor  
)    else:        return buscar(                raiz.direita, valor  
)
```

A eficiência de busca em uma ABB balanceada é $O(\log n)$, tornando-a ideal para operações de busca, inserção e remoção frequentes.

Balanceamento de Árvores

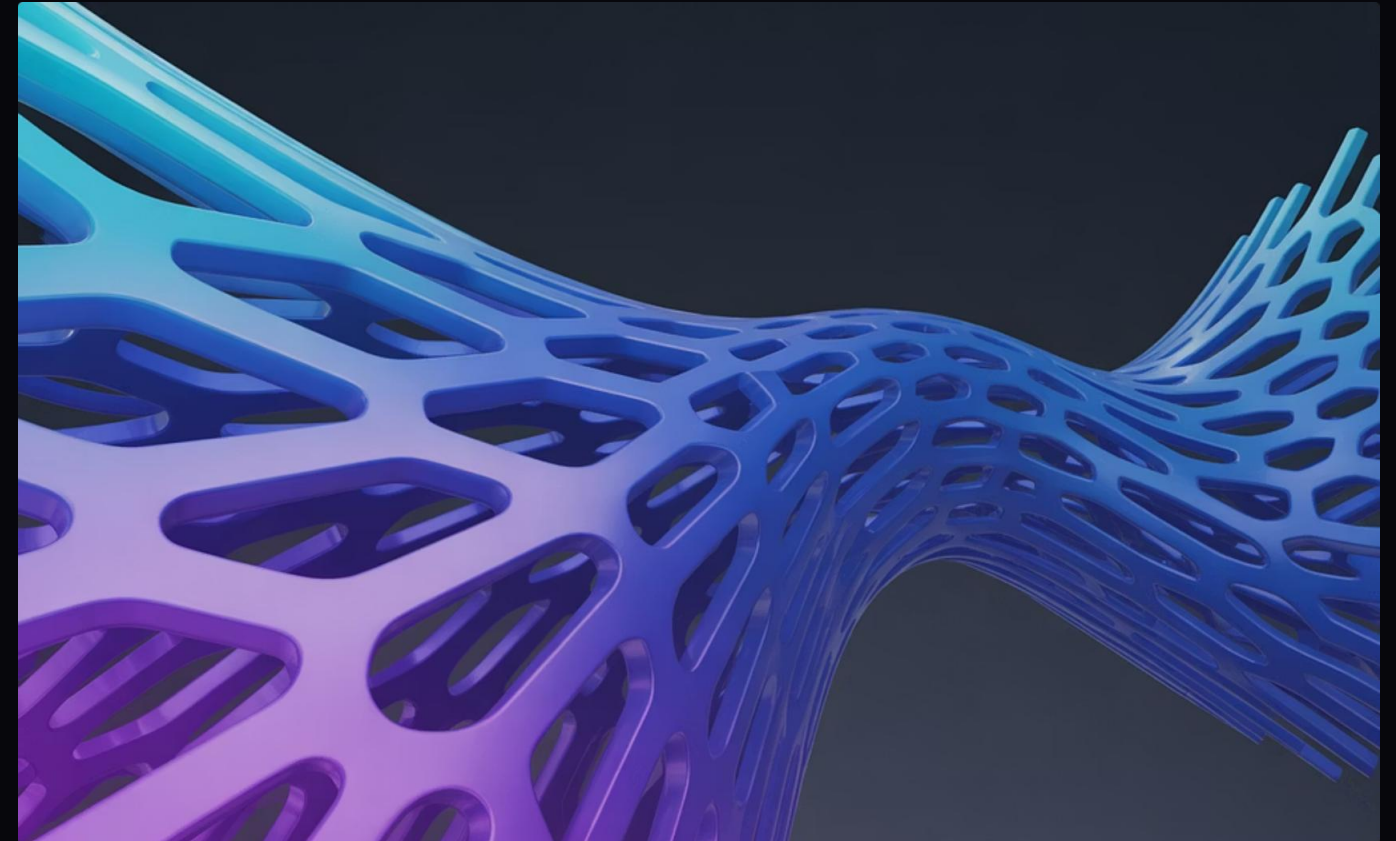


Árvore Balanceada

Alturas das subárvores diferem em no máximo 1, garantindo operações $O(\log n)$

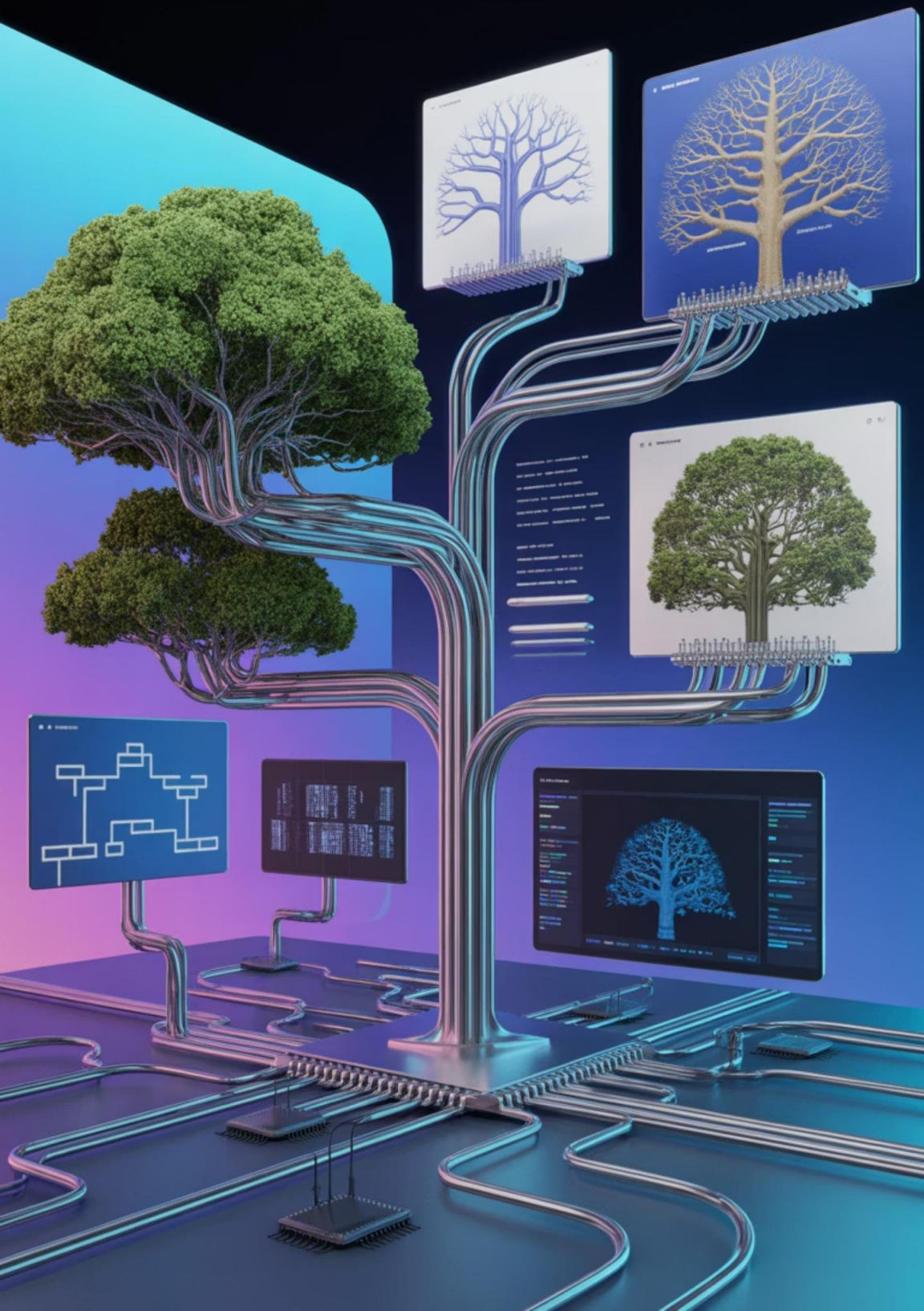
O balanceamento é crucial para manter a eficiência das operações. Árvores como **AVL** e **Rubro-Negras** implementam mecanismos automáticos de rebalanceamento através de rotações, garantindo que a altura permaneça logarítmica em relação ao número de nós.

Sem balanceamento, inserções sequenciais podem criar uma árvore que se comporta como uma lista encadeada, perdendo todas as vantagens de performance da estrutura hierárquica.



Árvore Degenerada

Estrutura linear que degrada performance para $O(n)$ nas operações



Capítulo 3: Aplicações Práticas de Árvores

As árvores transcendem a teoria, sendo fundamentais em inúmeros sistemas e aplicações do mundo real, desde sistemas operacionais até algoritmos de inteligência artificial.

Sistemas de Arquivos e Diretórios

Todo sistema operacional moderno organiza arquivos em uma estrutura hierárquica de árvore. O diretório raiz (/ no Unix/Linux ou C:\ no Windows) é a raiz da árvore, e cada diretório pode conter arquivos (folhas) e subdiretórios (nós internos).

Estrutura Típica Unix/Linux

```

/├─ home│   │─ usuario│   │   │─ documentos│   │   │   │─ imagens│   │   │   └─
downloads├─ etc│   │─ config│   │   └─ hosts├─ var│   │   │─ log│   │   └─ www└─ usr
├─ bin   └─ lib

```

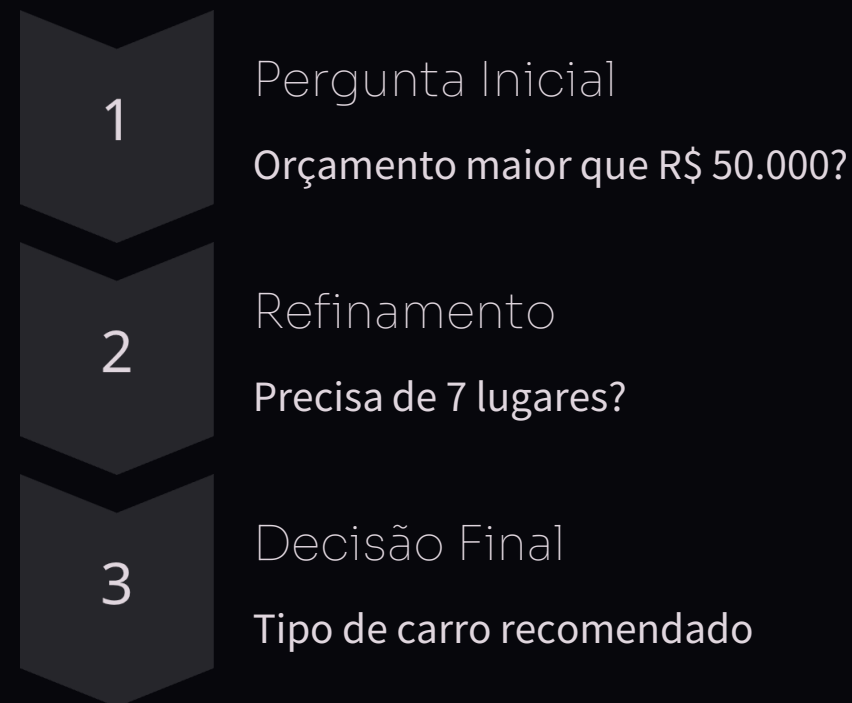
Percorrendo Diretórios em Python

```
import os
def listar_arvore(caminho, nivel=0):
    try:
        itens = os.listdir(caminho)
        for item in itens:
            print("  " * nivel + "└── " + item)
            caminho_completo = \
                os.path.join(
                    caminho, item
                )
            if os.path.isdir(
                caminho_completo
            ):
                listar_arvore(
                    caminho_completo,
                    nivel + 1
                )
    except PermissionError:
        pass
```

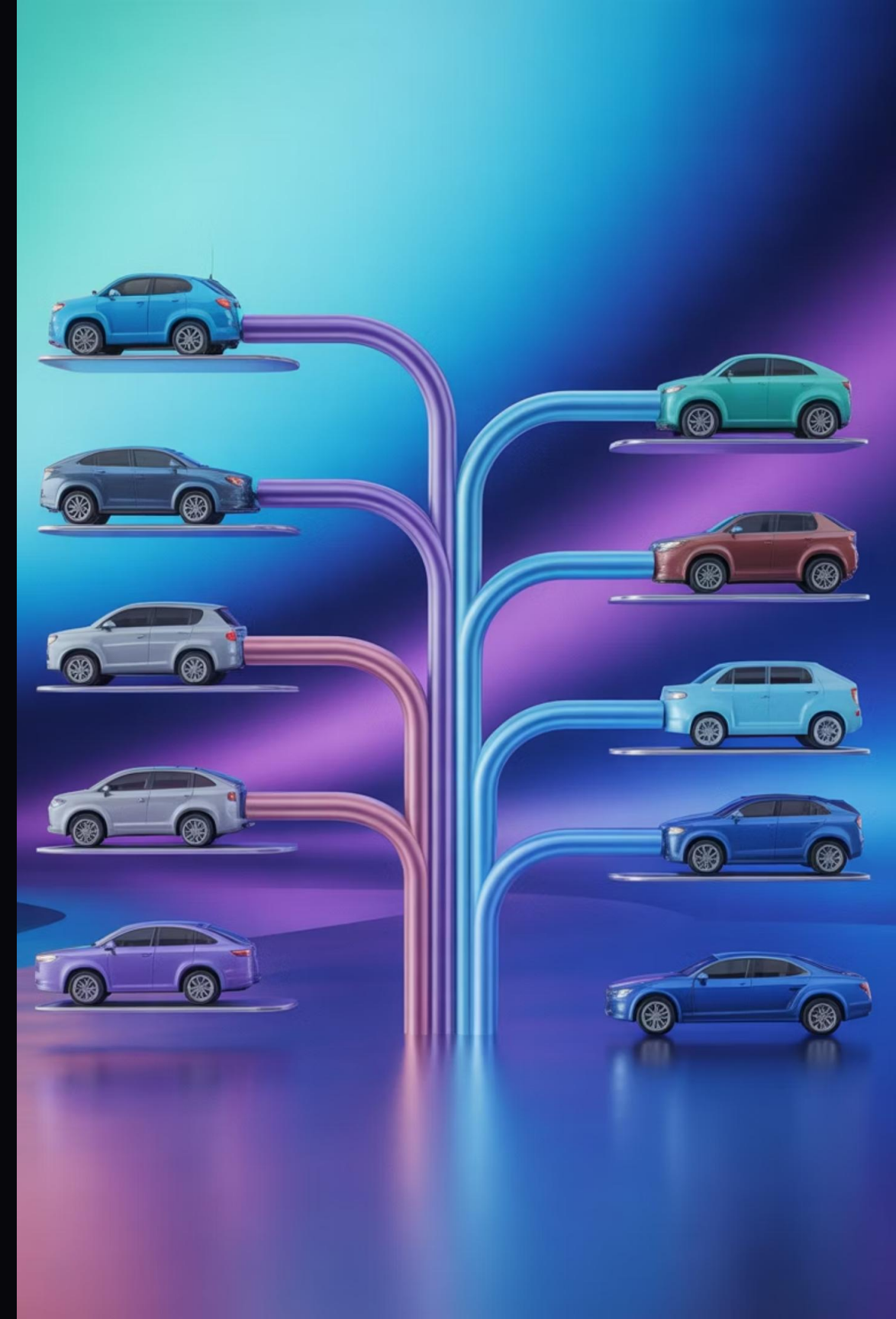
Esta aplicação demonstra como árvores modelam naturalmente hierarquias do mundo real, permitindo navegação eficiente e organização lógica de grandes volumes de dados.

Árvores de Decisão para Tomada de Decisão

Árvores de decisão são ferramentas poderosas para classificação e regressão em machine learning. Cada nó interno representa uma pergunta sobre um atributo, cada ramo representa uma resposta possível, e cada folha representa uma decisão final ou predição.



A simplicidade e interpretabilidade das árvores de decisão as tornam populares em aplicações onde a explicabilidade é importante, como diagnósticos médicos, aprovação de crédito e sistemas de recomendação.



Algoritmos de Busca e Ordenação

Busca Binária em ABB

A propriedade ordenada da ABB permite busca eficiente em $O(\log n)$, eliminando metade das possibilidades a cada comparação.

```
# Busca iterativa
def busca_iterativa(raiz, alvo):
    atual = raiz
    while atual:
        if alvo == atual.valor:
            return True
        elif alvo < atual.valor:
            atual = atual.esquerda
        else:
            atual = atual.direita
    return False
```

Heap para Filas de Prioridade

Heaps são árvores binárias completas onde cada nó é maior (max-heap) ou menor (min-heap) que seus filhos, permitindo acesso rápido ao elemento extremo.

```
import heapq # Min-heap em Python
fila = []
heapq.heappush(fila, 5)
heapq.heappush(fila, 2)
heapq.heappush(fila, 8)
menor = heapq.heappop(fila)
print(menor) # Saída: 2
```

Heaps são fundamentais em algoritmos como Heapsort e em implementações de filas de prioridade usadas em algoritmos de grafos como Dijkstra e A*.

Codificação de Huffman

A codificação de Huffman é um algoritmo de compressão que utiliza uma árvore binária para atribuir códigos de comprimento variável aos caracteres, onde caracteres mais frequentes recebem códigos mais curtos.

01

Calcular Frequências

Contar a ocorrência de cada caractere no texto de entrada

02

Criar Nós Folha

Cada caractere vira um nó com sua frequência como peso

03

Construir Árvore

Combinar os dois nós de menor frequência repetidamente

04

Gerar Códigos

Percorrer da raiz às folhas: esquerda=0, direita=1

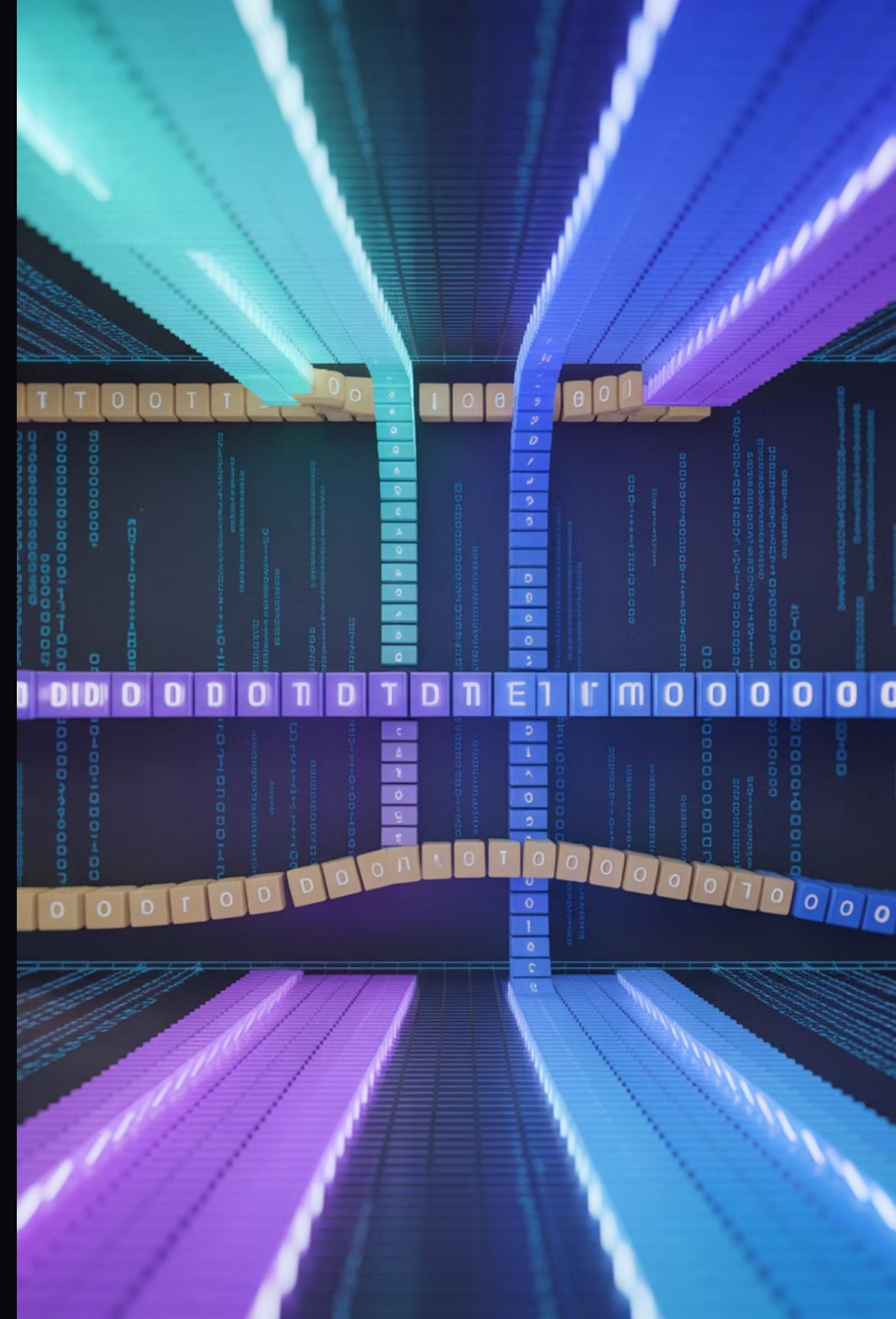
Exemplo Prático: Texto "ABRACADABRA"

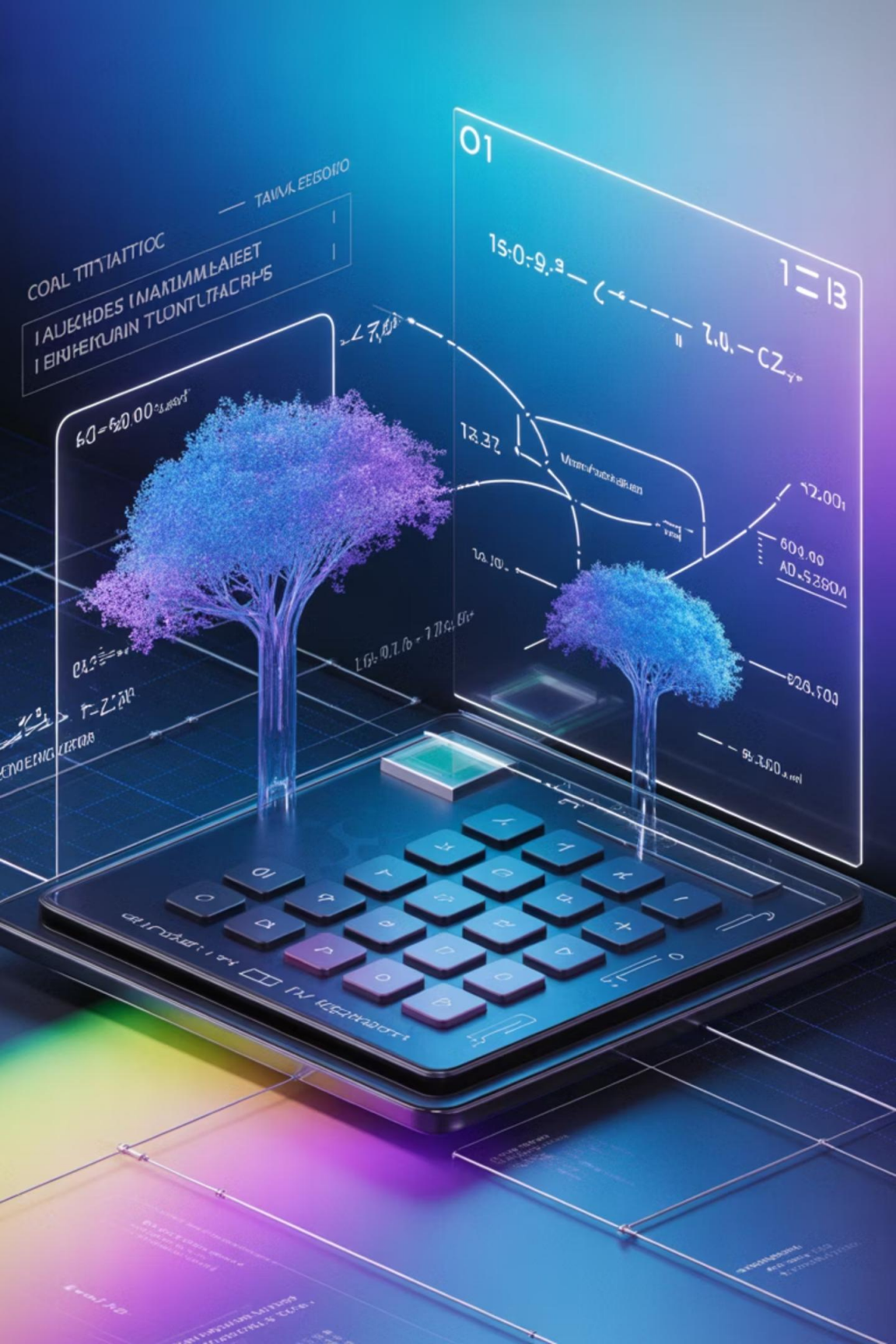
Frequências: A=5, B=2, R=2, C=1, D=1

Códigos gerados: A=0, B=110, R=10, C=1110, D=1111

Taxa de compressão: 88 bits (8×11) $\rightarrow$ 29 bits = 67% de redução

A árvore de Huffman organiza os caracteres de modo que os mais frequentes fiquem mais próximos da raiz, resultando em códigos mais curtos e maior compressão geral do texto.





Capítulo 4: Exemplos Práticos de Cálculo e Manipulação

Agora vamos trabalhar com exemplos concretos, calculando propriedades e manipulando árvores passo a passo para consolidar o entendimento prático.

Construção Manual de uma Árvore Binária

Vamos construir uma árvore binária completa do zero, inserindo os valores e calculando suas propriedades fundamentais.

Passo 1: Criar Raiz

Inserir valor 15 como raiz da árvore

Passo 2: Adicionar Nível 1

Inserir 10 à esquerda e 20 à direita da raiz

Passo 3: Adicionar Nível 2

Inserir 5, 12 (filhos de 10) e 18, 25 (filhos de 20)

Passo 4: Calcular Propriedades

Determinar altura (3), número de nós (7) e profundidade de cada nó

Altura de Cada Subárvore

- Raiz (15): altura 2
- Nó 10: altura 1
- Nó 20: altura 1
- Folhas: altura 0

Profundidade de Cada Nó

- Raiz (15): profundidade 0
- Nós 10, 20: profundidade 1
- Nós 5, 12, 18, 25: profundidade 2

Cálculo de Caminhamentos (Travessias)

Utilizando a árvore construída anteriormente (raiz 15), vamos calcular manualmente os resultados das três travessias principais e visualizar o processo de visitação.

Pré-Ordem (Raiz $\rightarrow$ Esq $\rightarrow$ Dir)

Resultado: 15, 10, 5, 12, 20, 18, 25

Visitamos cada nó antes de explorar suas subárvores

Em-Ordem (Esq $\rightarrow$ Raiz $\rightarrow$ Dir)

Resultado: 5, 10, 12, 15, 18, 20, 25

Produz sequência ordenada em ABB

Pós-Ordem (Esq $\rightarrow$ Dir $\rightarrow$ Raiz)

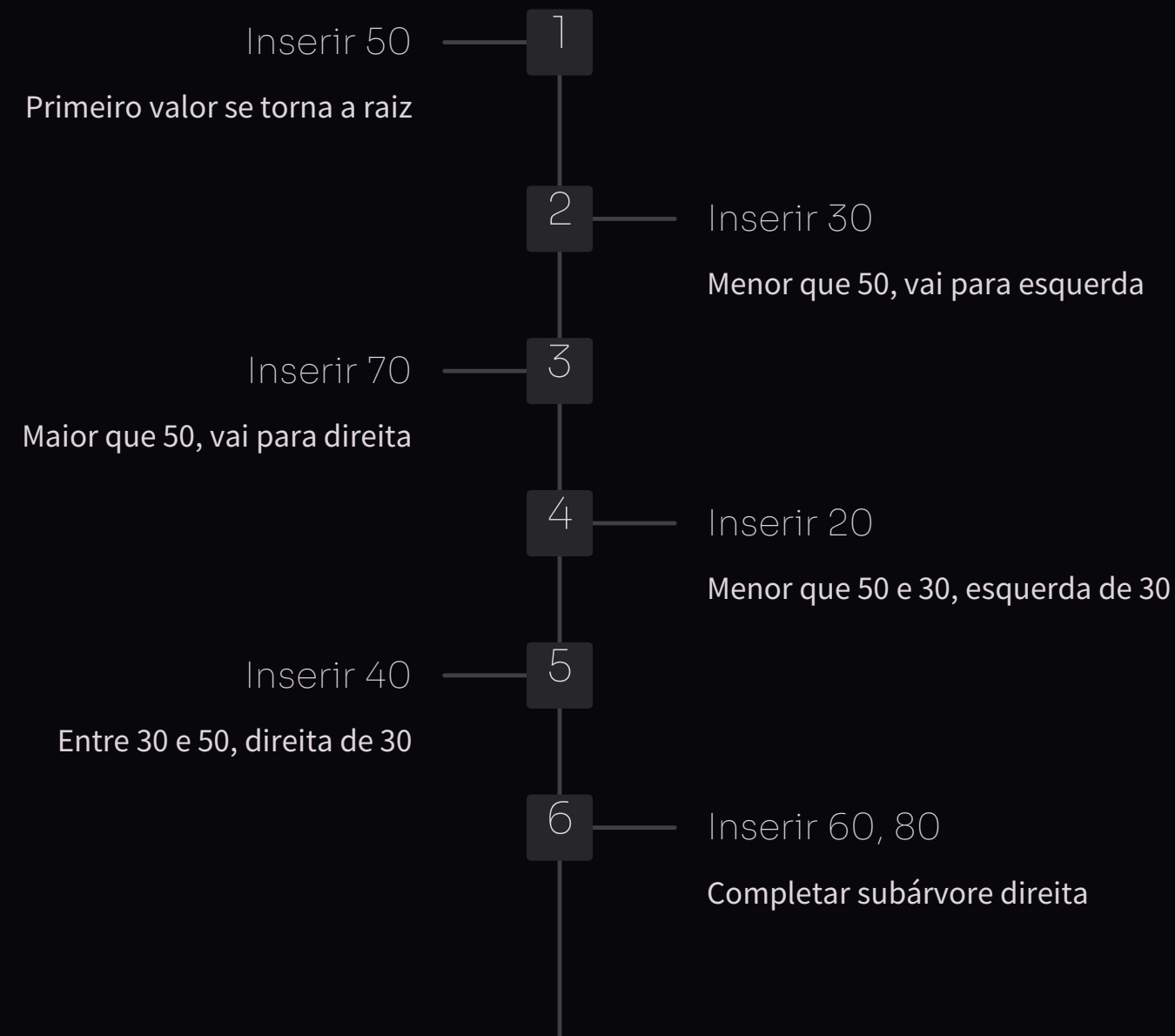
Resultado: 5, 12, 10, 18, 25, 20, 15

Visitamos nós após suas subárvores, útil para liberar memória

A travessia em-ordem é especialmente importante pois, em uma ABB, produz os elementos em ordem crescente. Já a pós-ordem é ideal para operações que dependem do processamento dos filhos primeiro, como cálculo de altura ou liberação de memória.

Construção de ABB a partir de uma Lista de Valores

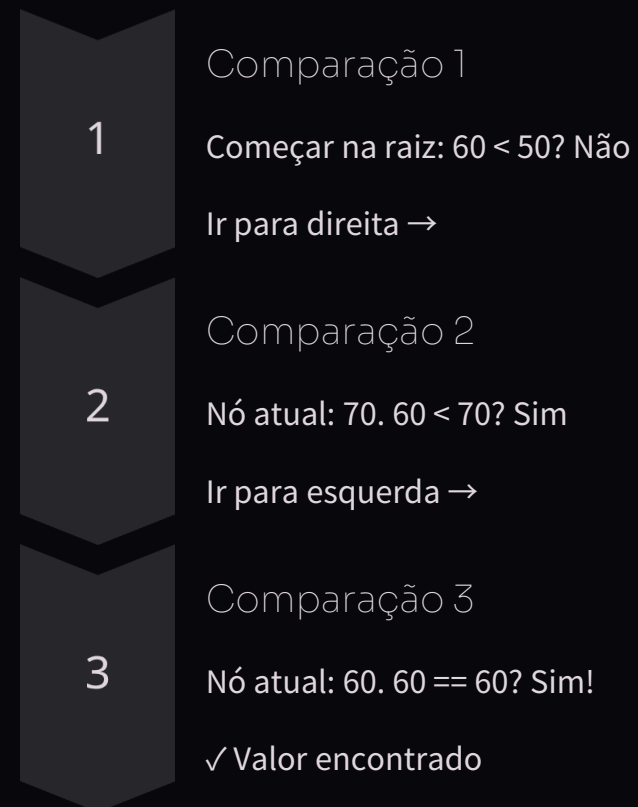
Vamos construir uma Árvore Binária de Busca inserindo sequencialmente os valores: 50, 30, 70, 20, 40, 60, 80



Árvore Resultante: Perfeitamente balanceada com altura 2, onde 50 é a raiz, 30 e 70 formam o segundo nível, e 20, 40, 60, 80 são as folhas no terceiro nível.

Busca em ABB: Passo a Passo

Vamos buscar o valor **60** na ABB construída anteriormente, documentando cada comparação e decisão tomada durante o processo.



Implementação Iterativa

```
def buscar_iterativo(raiz, alvo):  
    atual = raiz    passos = 0    while  
    atual:           passos += 1    print(f"Passo {passos}: "  
    f"Comparando {alvo} "         f"com {atual.valor}")        if  
    alvo == atual.valor:           print(f"Encontrado em "  
    f"{passos} passos!")          return True        elif alvo < atual.valor:  
    atual = atual.esquerda        else:            atual = atual.direita  
    return False
```

A busca em uma ABB balanceada requer $O(\log n)$ comparações, tornando-a extremamente eficiente mesmo para grandes volumes de dados.

Inserção e Remoção em ABB

A remoção é a operação mais complexa em ABBs, pois precisamos manter a propriedade de ordenação. Existem três casos distintos a considerar.

1	2	3
Remover Nó Folha Caso mais simples: apenas remover o nó, atualizando a referência do pai para None Exemplo: remover 20 da nossa ABB	Remover Nó com Um Filho Substituição direta: conectar o pai do nó removido diretamente ao seu único filho Exemplo: se 30 tivesse apenas o filho 20	Remover Nó com Dois Filhos Caso complexo: encontrar o sucessor in-order (menor valor da subárvore direita) ou predecessor (maior da esquerda), substituir valores e remover o sucessor/predecessor Exemplo: remover 50 (raiz)

Código para Remoção Completa

```
def remover(raiz, valor):
    if raiz is None:
        return raiz
    if valor < raiz.valor:
        raiz.esquerda = remover(raiz.esquerda, valor)
    elif valor > raiz.valor:
        raiz.direita = remover(raiz.direita, valor)
    else:
        # Caso 1 e 2: nó folha ou um filho
        if raiz.esquerda is None:
            return raiz.direita
        elif raiz.direita is None:
            return raiz.esquerda
        # Caso 3: dois filhos
        sucessor = encontrar_minimo(raiz.direita)
        raiz.valor = sucessor.valor
        raiz.direita = remover(raiz.direita, sucessor.valor)
    return raiz
```

Balanceamento Automático: Árvore AVL

Árvores AVL mantêm-se automaticamente balanceadas através de rotações após cada inserção ou remoção. O fator de balanceamento de cada nó (diferença de altura entre subárvores esquerda e direita) deve estar entre -1 e 1.

Tipos de Rotações

- 

Rotação Simples à Direita

Quando inserção na subárvore esquerda-esquerda causa desbalanceamento
- 

Rotação Simples à Esquerda

Quando inserção na subárvore direita-direita causa desbalanceamento
- 

Rotação Dupla Esq-Dir

Inserção na subárvore esquerda-direita
- 

Rotação Dupla Dir-Esq

Inserção na subárvore direita-esquerda

Exemplo: Inserir 25 na ABB

Árvore inicial: 50(raiz), 30, 70, 20, 40, 60, 80

Inserir 25 → vai entre 20 e 30

Problema: Nó 30 fica com fator +2 (desbalanceado)

Solução: Rotação dupla esquerda-direita no nó 30

1. Rotação esquerda em 20
2. Rotação direita em 30
3. Resultado: 25 se torna novo pai de 20 e 30

AVL garante altura $O(\log n)$, mas o custo de manutenção pode ser alto em aplicações com muitas inserções/remoções. Para esses casos, árvores rubro-negras são preferíveis.

Árvores Rubro-Negras: Conceitos e Aplicações

Árvores rubro-negras são uma alternativa às AVL, oferecendo balanceamento menos rígido mas com menor custo de manutenção. Cada nó tem uma cor (vermelho ou preto) que ajuda a manter a estrutura aproximadamente balanceada.

Propriedade 1

Todo nó é vermelho ou preto

Propriedade 2

A raiz é sempre preta

Propriedade 3

Todas as folhas (NIL) são pretas

Propriedade 4

Nós vermelhos têm filhos pretos

Propriedade 5

Caminhos têm mesmo número de nós pretos

Estas propriedades garantem que o caminho mais longo é no máximo duas vezes o caminho mais curto, mantendo operações em $O(\log n)$. Árvores rubro-negras são usadas em implementações de map/set em C++, Java e TreeMap em Python.

Árvores em Estruturas de Dados Avançadas



Trie (Árvore de Prefixos)

Estrutura especializada para armazenamento e busca eficiente de strings. Cada nó representa um caractere, e caminhos da raiz formam palavras completas.

Aplicações: Autocomplete, correção ortográfica, roteamento IP



B-Trees

Árvores balanceadas onde cada nó pode ter múltiplos filhos e chaves. Projetadas para minimizar acessos a disco em sistemas de arquivos e bancos de dados.

Aplicações: Índices de banco de dados, sistemas de arquivos (ext4, NTFS)



B+ Trees

Variante de B-Tree onde dados ficam apenas nas folhas, conectadas em lista. Ideal para consultas de intervalo e varreduras sequenciais.

Aplicações: Índices primários em SGBDs, armazenamento de registros

Uma Trie armazenando as palavras "casa", "carro", "cachorro" e "dado" demonstra como prefixos comuns compartilham caminhos, economizando espaço e acelerando buscas.



Árvores em Machine Learning: Árvores de Decisão

Árvores de decisão são modelos de aprendizado supervisionado que dividem o espaço de características recursivamente, criando regiões onde a predição é constante. São a base de algoritmos poderosos como Random Forests e Gradient Boosting.

Exemplo: Classificação de Íris

```
from sklearn.tree import \    DecisionTreeClassifierfrom sklearn.datasets
import \    load_irisimport matplotlib.pyplot as plt# Carregar dadosiris
= load_iris()X, y = iris.data, iris.target# Treinar modeloclf =
DecisionTreeClassifier(    max_depth=3)clf.fit(X, y)#
Acuráciaprint(f"Acurácia: "    f"{clf.score(X, y):.2f}")
```

Visualização da Árvore

```
from sklearn.tree import \
plot_treeplt.figure(figsize=(15,10))plot_tree(    clf,
feature_names=iris.feature_names,    class_names=iris.target_names,
filled=True)plt.show()
```

A árvore resultante mostra como o modelo divide o espaço de características (comprimento e largura de sépalas e pétalas) para classificar as três espécies de íris.

Cálculo do Ganho de Informação em Árvores de Decisão

O ganho de informação mede quanto uma divisão reduz a incerteza (entropia) nos dados. O algoritmo escolhe o atributo com maior ganho para cada nó.

01

Calcular Entropia Inicial

$H(S) = -\sum p_i \log_2(p_i)$, onde p_i é a proporção da classe i

03

Calcular Ganho

$\text{Ganho} = H(S) - H(S|\text{Atributo})$

Exemplo Numérico

Dataset: 10 exemplos, 6 positivos (+), 4 negativos (-)

Entropia inicial: $H(S) = -(6/10)\log_2(6/10) - (4/10)\log_2(4/10) = 0.971$

Após divisão por atributo A:

- Ramo esquerdo: 5 exemplos (4+, 1-) $\rightarrow H = 0.722$
- Ramo direito: 5 exemplos (2+, 3-) $\rightarrow H = 0.971$
- $H(S|A) = (5/10) \times 0.722 + (5/10) \times 0.971 = 0.846$

Ganho de Informação: $0.971 - 0.846 = 0.125$

02

Para Cada Atributo

Calcular entropia ponderada após divisão

04

Escolher Melhor Divisão

Atributo com maior ganho de informação

Aplicação Real: Diagnóstico Médico com Árvore de Decisão

Árvores de decisão são amplamente usadas em medicina para apoio ao diagnóstico, onde sintomas observados levam a possíveis condições através de uma série de perguntas estruturadas.



Exemplo de Código Simplificado:

```
def diagnosticar(sintomas): if sintomas['febre'] >= 38: if sintomas['dor_garganta']: if sintomas['tosse']: return "Possível gripe - consultar médico" else: return "Possível infecção bacteriana" else: return "Monitorar temperatura" else: return "Repouso e hidratação"
```

Árvores para Representação de Expressões Matemáticas

Expressões aritméticas podem ser representadas como árvores binárias, onde operadores são nós internos e operandos são folhas. Esta representação facilita avaliação e conversão entre notações.

Expressão: $(3 + 5) \times (7 - 2)$

A árvore correspondente tem:

- **Raiz:** operador $\times$ (multiplicação)
- **Filho esquerdo de $\times$:** $+$ com filhos 3 e 5
- **Filho direito de $\times$:** $-$ com filhos 7 e 2

Esta estrutura preserva a ordem de precedência e permite diferentes tipos de travessia para gerar diferentes notações.

Avaliação Pós-Ordem

```
def avaliar(no):    if no.eh_folha():        return no.valor
esq = avaliar(no.esquerda)    dir = avaliar(no.direita)    if
no.operador == '+':        return esq + dir    elif no.operador ==
'-':        return esq - dir    elif no.operador == '*':
return esq * dir    elif no.operador == '/':        return esq /
dir
```

Resultado: $(3+5) \times (7-2) = 8 \times 5 = 40$

Construção de Árvore a partir de Expressão Infixa

Converter uma expressão matemática em notação infix (padrão) para árvore requer parsing cuidadoso respeitando precedência de operadores e parênteses.



Tokenização

Quebrar expressão em tokens: números, operadores, parênteses



Usar Pilhas

Manter pilha de operadores e pilha de operandos/subárvores



Respeitar Precedência

Processar operadores de maior precedência primeiro



Construir Árvore

Combinar operandos com operadores para formar subárvores

Código Simplificado

```
def construir_arvore expressao(expressao): # Precedência de operadores precedencia = {'+': 1, '-': 1, '*': 2, '/': 2}
pilha_operadores = [] pilha_operandos = [] for token in tokenizar(expressao): if token.isdigit():
pilha_operandos.append(NoArvore(int(token))) elif token in precedencia: while (pilha_operadores and
precedencia[pilha_operadores[-1]] >= precedencia[token]): processar_operador(pilha_operadores, pilha_operandos)
pilha_operadores.append(token) # ... processar parênteses ... while pilha_operadores:
processar_operador(pilha_operadores, pilha_operandos) return pilha_operandos[0]
```

[illegible]



Árvores para Jogos e Inteligência Artificial

Em jogos de estratégia, árvores de jogo representam todas as possíveis sequências de movimentos. Algoritmos como minimax e poda alfa-beta exploram essas árvores para encontrar a melhor jogada.

1

Gerar Jogadas Possíveis

Cada nó representa um estado do jogo após um movimento

2

Avaliar Estados Folha

Atribuir pontuação: +1 vitória, 0 empate, -1 derrota

3

Propagar Valores (Minimax)

Jogador maximiza sua pontuação, oponente minimiza

4

Escolher Melhor Movimento

Seguir caminho que leva ao melhor resultado garantido

Poda Alfa-Beta: Otimização que corta ramos da árvore que comprovadamente não podem levar a uma solução melhor, reduzindo drasticamente o número de estados a avaliar.

Árvores para Indexação em Bancos de Dados

Bancos de dados usam B-Trees e B+ Trees para criar índices que permitem busca, inserção e remoção rápidas, mesmo com milhões de registros. A estrutura minimiza acessos a disco.

B-Tree: Características

- Cada nó contém múltiplas chaves (ordem m)
- Nós podem ter entre $m/2$ e m filhos
- Todas as folhas no mesmo nível
- Chaves ordenadas em cada nó

Exemplo: B-Tree de ordem 3

Nó interno: $[10, 20]$ com 3 filhos

- Filho 1: valores < 10
- Filho 2: valores entre 10 e 20
- Filho 3: valores > 20

B+ Tree: Vantagens Adicionais

- Dados apenas nas folhas
- Folhas conectadas em lista
- Nós internos apenas para navegação
- Ideal para queries de intervalo

Operação de busca:

```
SELECT * FROM tabela WHERE idade BETWEEN 25 AND 35
```

O B+ Tree permite navegar até 25, depois percorrer as folhas sequencialmente até 35, sem subir e descer na árvore.

Árvores para Compressão de Dados

Revisitando a codificação de Huffman, vamos implementar um exemplo completo de compressão e descompressão de texto usando árvores.

Implementação Completa em Python

```
import heapq
from collections import defaultdict

class NoHuffman:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.esquerda = None
        self.direita = None

    def __lt__(self, other):
        return self.freq < other.freq

def construir_arvore_huffman(texto):
    # Calcular frequências
    freq = defaultdict(int)
    for char in texto:
        freq[char] += 1

    # Criar heap com nós folha
    heap = [NoHuffman(char, f) for char, f in freq.items()]
    heapq.heapify(heap)

    # Construir árvore
    while len(heap) > 1:
        esq = heapq.heappop(heap)
        dir = heapq.heappop(heap)
        interno = NoHuffman(None, esq.freq + dir.freq)
        interno.esquerda = esq
        interno.direita = dir
        heapq.heappush(heap, interno)

    return heap[0]

def gerar_codigos(raiz, codigo_atual="", codigos={}):
    if raiz.char:
        # É folha
        codigos[raiz.char] = codigo_atual
        return codigos

    gerar_codigos(raiz.esquerda, codigo_atual + "0", codigos)
    gerar_codigos(raiz.direita, codigo_atual + "1", codigos)

    return codigos
```

Árvores em Redes e Protocolos

Árvores geradoras (spanning trees) são subgrafos de uma rede que conectam todos os nós sem formar ciclos. São fundamentais em protocolos de roteamento e design de redes eficientes.

Algoritmo de Prim

Constrói árvore geradora mínima começando de um vértice e sempre adicionando a aresta de menor custo que conecta a árvore a um novo vértice

Algoritmo de Kruskal

Ordena todas as arestas por peso e adiciona uma por vez à árvore, desde que não forme ciclo

Exemplo: Rede com 5 Cidades

Cidades: A, B, C, D, E

Custos de conexão: AB=4, AC=2, AD=5, BC=1, BD=3, CD=4, CE=3, DE=2

Árvore Geradora Mínima (Prim):

1. Começar em A, adicionar AC (custo 2) - Total: 2
2. Adicionar CB (custo 1) - Total: 3
3. Adicionar BD (custo 3) - Total: 6
4. Adicionar DE (custo 2) - Total: 8

Custo total mínimo: 8 unidades



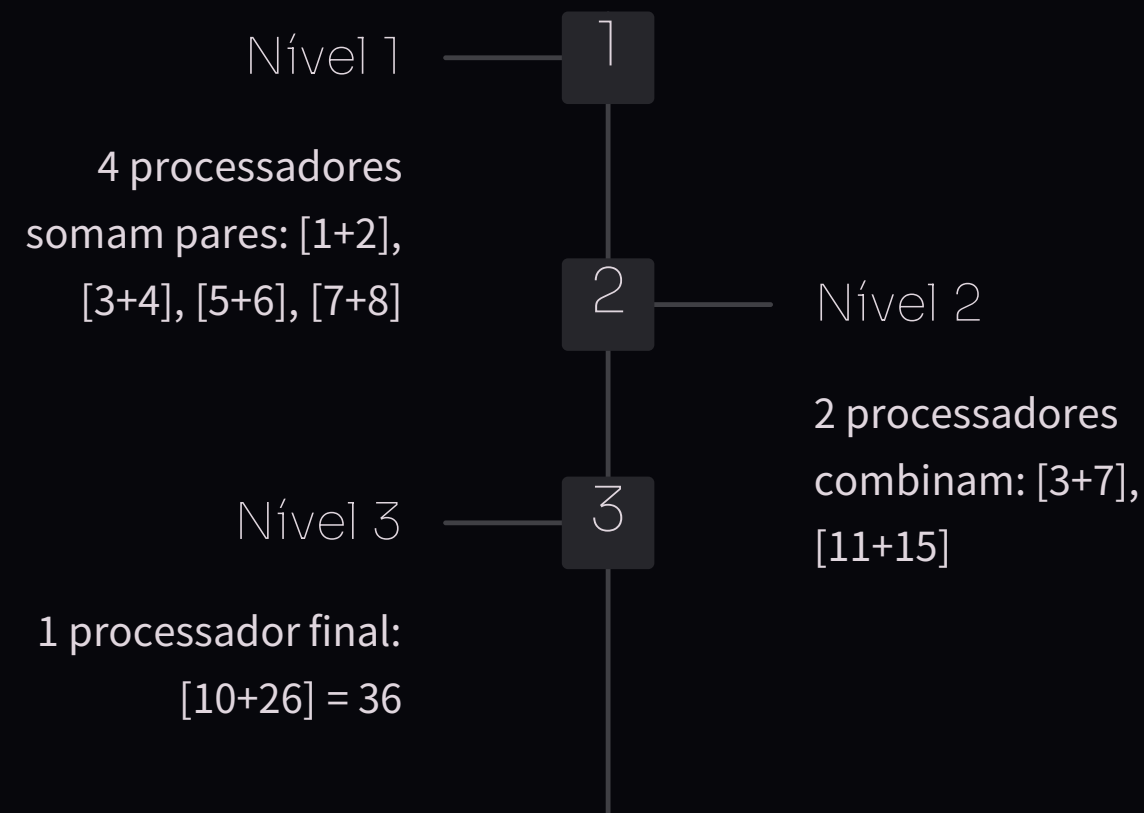
Visualização de um grafo completo com a árvore geradora mínima destacada, mostrando como é possível conectar todos os nós com o menor custo total possível.

Árvores em Estruturas de Dados Paralelas

Em computação paralela, árvores binárias são usadas para operações de redução, onde múltiplos processadores combinam resultados de forma hierárquica, reduzindo o tempo total de processamento.

Redução Paralela: Soma

Calcular soma de 8 números usando 4 processadores:



Benefícios e Desafios

Vantagens:

- Tempo logarítmico: $O(\log n)$
- Escalabilidade com mais processadores
- Eficiente para grandes volumes de dados

Desafios:

- Overhead de comunicação entre processos
- Sincronização necessária em cada nível
- Balanceamento de carga entre processadores

A estrutura em árvore garante que nenhum processador fique ocioso enquanto outros trabalham, maximizando a eficiência paralela.

Árvores em Linguagens de Programação

Compiladores e interpretadores usam Árvores Sintáticas Abstratas (AST) para representar a estrutura do código fonte. Cada nó representa uma construção da linguagem, facilitando análise, otimização e geração de código.



Análise Léxica

Quebra o código em tokens (palavras-chave, identificadores, operadores)



Análise Sintática

Constrói AST verificando que tokens seguem gramática da linguagem



Análise Semântica

Verifica tipos, escopo de variáveis navegando pela AST



Otimização

Transforma AST eliminando redundâncias e simplificando expressões



Geração de Código

Percorre AST gerando código de máquina ou bytecode

Implementação de Árvore Sintática em Python

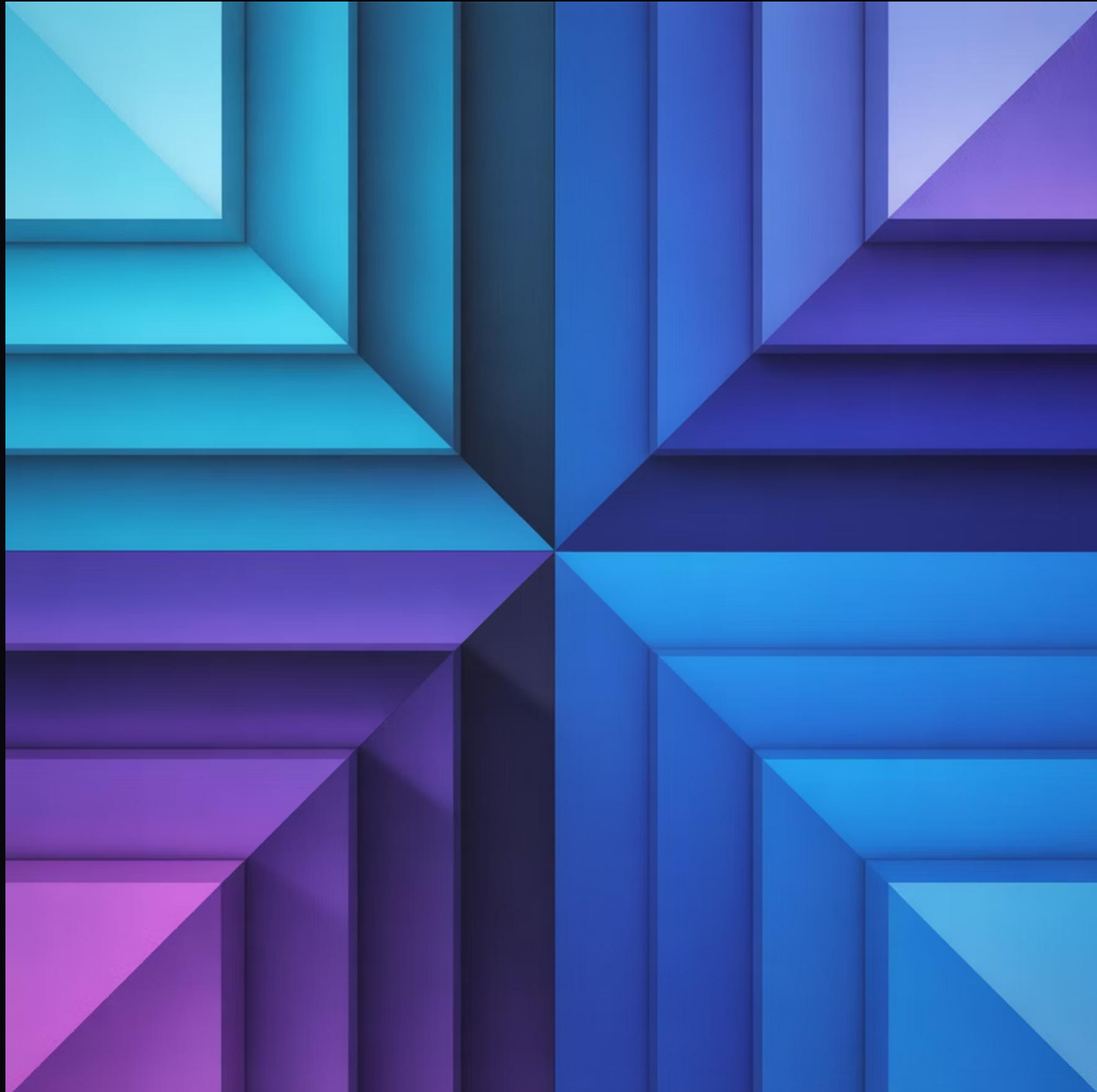
Vamos criar uma AST simples para expressões matemáticas e usá-la para avaliar e imprimir a expressão em diferentes formatos.

```
class NoAST: pass
class Numero(NoAST):
    def __init__(self, valor):
        self.valor = valor
    def avaliar(self):
        return self.valor
    def __str__(self):
        return str(self.valor)
class Operacao(NoAST):
    def __init__(self, op, esquerda, direita):
        self.op = op
        self.esquerda = esquerda
        self.direita = direita
    def avaliar(self):
        esq = self.esquerda.avaliar()
        dir = self.direita.avaliar()
        if self.op == '+':
            return esq + dir
        if self.op == '-':
            return esq - dir
        if self.op == '*':
            return esq * dir
        if self.op == '/':
            return esq / dir
    def __str__(self):
        return f"({self.esquerda} {self.op} {self.direita})"
# Construir AST para: (3 + 5) * 2
ast = Operacao('*', Operacao('+', Numero(3), Numero(5)), Numero(2))
print(f"Expressão: {ast}") # (3 + 5) * 2
print(f"Resultado: {ast.avaliar()}") # 16
```

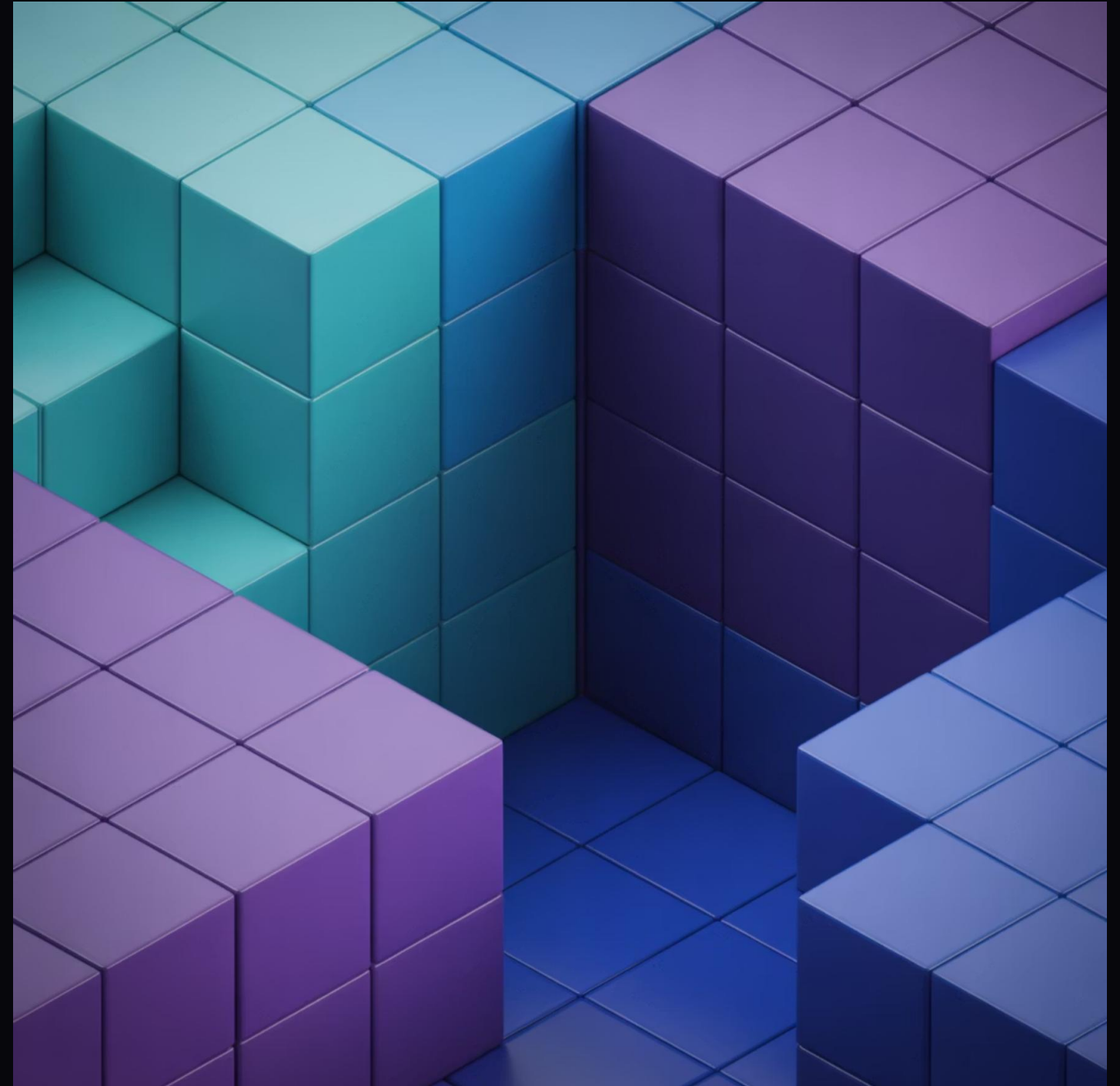
Árvores para Representação de Dados Geográficos

Quadtrees e Octrees dividem recursivamente o espaço em regiões menores, permitindo consultas espaciais eficientes como "encontrar todos os pontos dentro de uma área" ou detecção de colisão.

Quadtree (2D)



Octree (3D)



Árvores para Indexação de Texto

Estruturas especializadas como Suffix Trees e Tries permitem busca extremamente rápida de padrões em textos grandes, fundamentais em bioinformática e processamento de linguagem natural.



Trie

Armazena strings compartilhando prefixos.
Busca em $O(m)$ onde m é o tamanho da string.

Exemplo: Busca em Trie



Suffix Tree

Armazena todos os sufixos de um texto.
Permite busca de substring em $O(m)$.



Suffix Array

Array ordenado de sufixos.
Mais compacto que suffix tree.

```
class NoTrie: def __init__(self): self.filhos = {} self.fim_palavra = False
class Trie: def __init__(self): self.raiz = NoTrie()
def inserir(self, palavra): no = self.raiz for char in palavra: if char not in no.filhos: no.filhos[char] = NoTrie() no = no.filhos[char]
no.fim_palavra = True
def buscar(self, palavra): no = self.raiz for char in palavra: if char not in no.filhos: return False no = no.filhos[char]
return no.fim_palavra
# Usotrie = Trie()
for palavra in ["casa", "carro", "cachorro", "cama"]:
    trie.inserir(palavra)
print(trie.buscar("casa")) # True
print(trie.buscar("casas")) # False
```

Árvores em Sistemas de Arquivos Modernos

Sistemas de arquivos organizam diretórios e arquivos em estrutura de árvore, permitindo organização hierárquica e navegação eficiente através de comandos específicos.

Comandos Básicos (Unix/Linux)

```
# Listar árvore de diretórios
tree /home/usuario# Navegar
cd /home/usuario/documentos#
Criar estrutura
mkdir -p projetos/2024/janeiro# Copiar árvore inteira
cp -r origem/destino# Encontrar arquivos
find /home -name "*.txt"# Tamanho de diretório
du -sh /home/usuario
```

Navegação Programática

```
import os
def contar_arquivos(raiz):
    total = 0
    for dir, subdirs, arqs in \
        os.walk(raiz):
        total += len(arqs)
    return total
def encontrar_grandes(raiz, tamanho_mb):
    grandes = []
    for dir, _, arqs in \
        os.walk(raiz):
        for arq in arqs:
            caminho = os.path.join(
                dir, arq )
            tamanho = os.path.getsize(
                caminho )
            if tamanho > \
                tamanho_mb * 1024 * 1024:
                grandes.append(caminho)
    return grandes
```


Árvores para Representação de Organogramas e Hierarquias

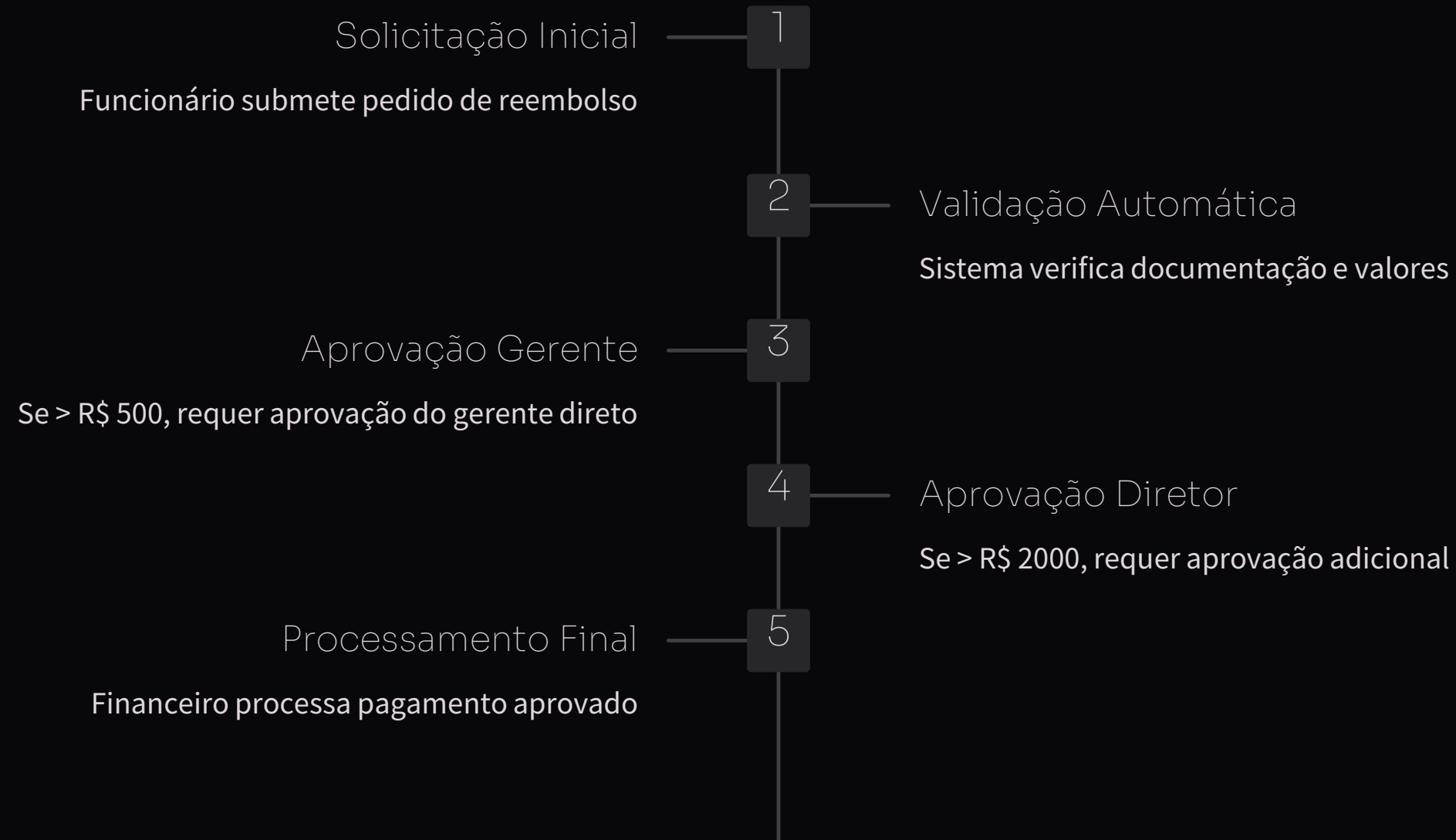
Organizações empresariais naturalmente se organizam em hierarquias, tornando árvores a estrutura ideal para representar relações de autoridade e responsabilidade.



Esta representação facilita análises como: calcular número de subordinados diretos/indiretos, identificar caminhos de comunicação, e planejar sucessões.

Árvores para Modelagem de Fluxos de Trabalho

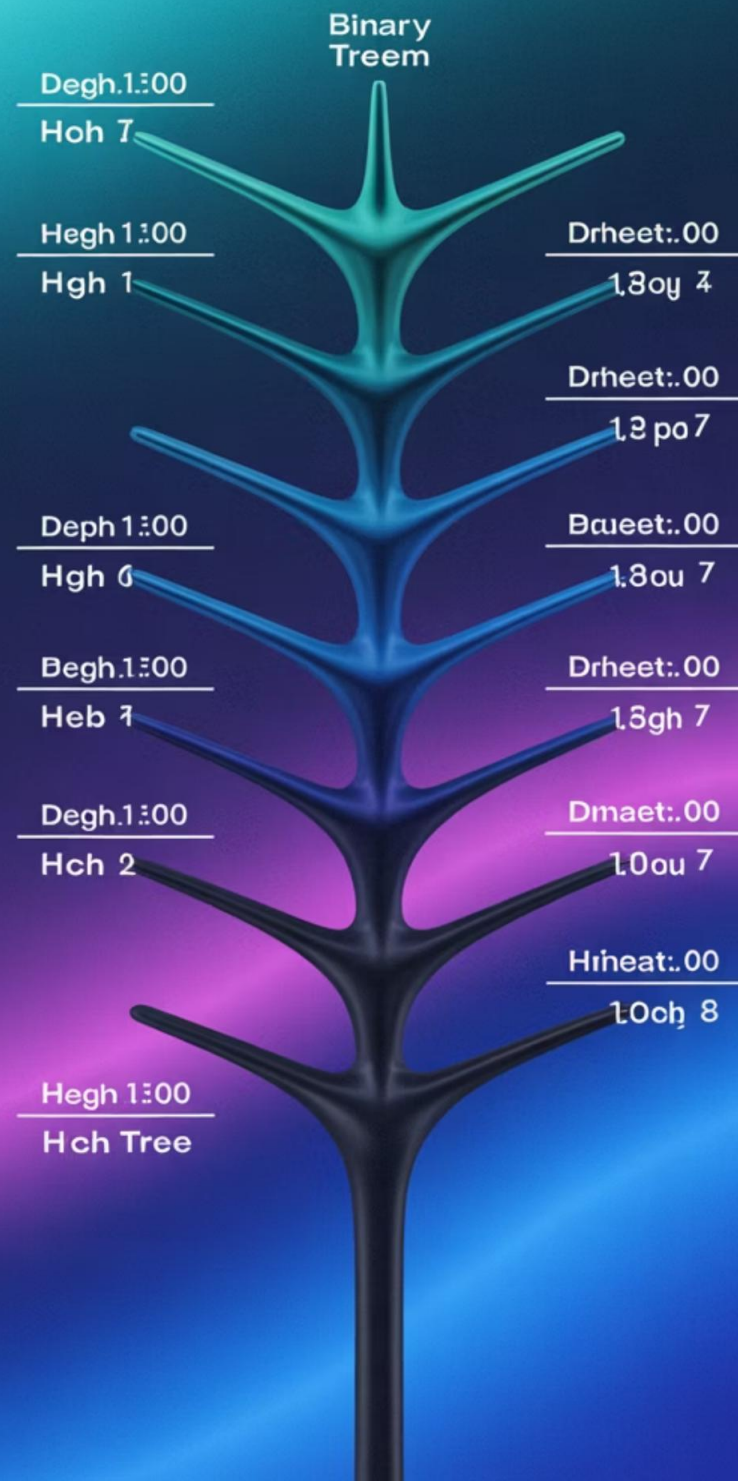
Processos de negócios com pontos de decisão são naturalmente modelados como árvores, onde cada nó de decisão leva a diferentes caminhos de execução.



Cada ponto de decisão na árvore representa uma condição que direciona o fluxo para diferentes ramos, permitindo automação e rastreamento eficientes do processo.

Fluxograma de decisão representado como árvore mostrando múltiplos caminhos possíveis para um processo de aprovação, com diferentes níveis de autoridade baseados no valor envolvido.





Exemplos de Cálculo: Altura, Profundidade e Grau

Agora vamos consolidar o aprendizado com exemplos detalhados de cálculo das propriedades mais importantes de árvores: altura, profundidade e grau de cada nó.

Cálculo da Altura da Árvore

A altura de uma árvore é o comprimento do caminho mais longo da raiz até uma folha. É uma métrica crucial para analisar a eficiência de operações na árvore.

Definição Formal

Para um nó n :

- Se n é folha: $\text{altura}(n) = 0$
- Caso contrário: $\text{altura}(n) = 1 + \max(\text{altura}(\text{filhos}))$

Código Recursivo

```
def calcular_altura(no):    if no is None:        return -1 # Árvore vazia
if no.esquerda is None and \    no.direita is None:        return 0 # Folha
altura_esq = calcular_altura(    no.esquerda    )    altura_dir =
calcular_altura(        no.direita        )    return 1 + max(altura_esq,
altura_dir)
```

Exemplo com Árvore

Árvore: 50(raiz), 30, 70, 20, 40, 60, 80

Cálculo bottom-up:

1. Folhas (20, 40, 60, 80): altura = 0
2. Nó 30: $\max(0, 0) + 1 = 1$
3. Nó 70: $\max(0, 0) + 1 = 1$
4. Raiz 50: $\max(1, 1) + 1 = 2$

A altura da árvore é 2, indicando 3 níveis (0, 1, 2).

Complexidade: $O(n)$ - visita cada nó uma vez

Cálculo do Grau de um Nó

O grau de um nó é simplesmente o número de filhos diretos que ele possui. É uma propriedade local que não requer recursão para calcular.

2

Raiz (50)

Possui dois filhos: 30 e 70

2

Nó 30

Possui dois filhos: 20 e 40

2

Nó 70

Possui dois filhos: 60 e 80

0

Folhas

20, 40, 60, 80 não possuem filhos

Código para Calcular Grau

```
def calcular_grau(no):  
    if no is None:  
        return 0  
    grau = 0  
    if no.esquerda is not None:  
        grau += 1  
    if no.direita is not None:  
        grau += 1  
    return graudef grau_maximo_arvore(raiz):  
    if raiz is None:  
        return 0  
    grau_atual = calcular_grau(raiz)  
    grau_esq = grau_maximo_arvore(raiz.esquerda)  
    grau_dir = grau_maximo_arvore(raiz.direita)  
    return max(grau_atual, grau_esq, grau_dir)
```

Para árvores binárias, o grau máximo é sempre 2. Para árvores gerais, pode ser qualquer valor.

Cálculo da Profundidade de um Nó

A profundidade de um nó é a distância (número de arestas) da raiz até esse nó. Diferente da altura, que olha para baixo, a profundidade olha para cima na hierarquia.

Abordagem com Busca

```
def calcular_profundidade(raiz, alvo, prof=0):
    if raiz is None:
        return -1 # Não encontrado
    if raiz.valor == alvo:
        return prof # Encontrado
    # Buscar na esquerda
    prof_esq = \
        calcular_profundidade(
            raiz.esquerda,
            alvo,
            prof + 1
        )
    if prof_esq != -1:
        return prof_esq # Encontrado na esquerda
    # Buscar na direita
    return \
        calcular_profundidade(
            raiz.direita,
            alvo,
            prof + 1
        )
```

Profundidades da Nossa Árvore

50	0	0
30, 70	1	1
20, 40	2	2
60, 80	2	2

Observação: Profundidade e nível são equivalentes - ambos contam a distância da raiz.

Exercício Guiado: Construção e Cálculo Completo

Vamos consolidar todo o aprendizado construindo uma árvore do zero e calculando todas as suas propriedades importantes.

Dados: Inserir na ordem: 45, 25, 65, 15, 35, 55, 75, 10, 30, 50, 70

01	02	03
Construir ABB	Calcular Altura Total	Listar Profundidades
Inserir valores seguindo propriedade de ordenação	Aplicar fórmula recursiva da raiz	Calcular distância de cada nó até a raiz
04	05	
Identificar Graus	Realizar Travessias	
Contar filhos de cada nó	Gerar sequências pré, em e pós-ordem	
Resultados	Interpretação	
Altura da árvore: 3	A árvore tem altura 3, indicando profundidade moderada. A travessia em- ordem confirma a propriedade de ABB (valores ordenados).	
Número de nós: 11		
Número de folhas: 4 (10, 30, 50, 70)	A maioria dos nós tem grau 2, indicando árvore bem preenchida. As 4 folhas estão distribuídas nos níveis 2 e 3.	
Grau máximo: 2	Esta configuração oferece bom equilíbrio entre profundidade e largura, resultando em operações eficientes $O(\log n)$ para busca, inserção e remoção.	
Pré-ordem: 45, 25, 15, 10, 35, 30, 65, 55, 50, 75, 70		
Em-ordem: 10, 15, 25, 30, 35, 45, 50, 55, 65, 70, 75		
Pós-ordem: 10, 15, 30, 35, 25, 50, 55, 70, 75, 65, 45		